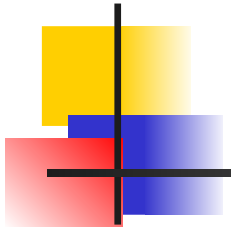


Data Mining: Concepts and Techniques

— Slides for Textbook —
— Chapter 6 —

©Jiawei Han and Micheline Kamber
Intelligent Database Systems Research Lab
School of Computing Science
Simon Fraser University, Canada
http://www.cs.sfu.ca

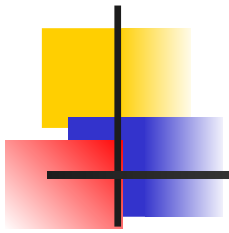
Chapter 6: Mining Association Rules in Large Databases

- Association rule mining
- Mining single-dimensional Boolean association rules from transactional databases
- Mining multilevel association rules from transactional databases
- Mining multidimensional association rules from transactional databases and data warehouse
- From association mining to correlation analysis
- Constraint-based association mining
- Summary

What Is Association Mining?

- Association rule mining:
  - Finding frequent patterns, associations, correlations, or causal structures among sets of items or objects in transaction databases, relational databases, and other information repositories.

- Applications:
  - Basket data analysis, cross-marketing, catalog design, loss-leader analysis, clustering, classification, etc.



Examples.

Rule form: "Body $\rightarrow$ Head [support, confidence]".

buys(x, "computers") $\rightarrow$ buys(x, financial_mgmt_s/w")

[2%, 60%]

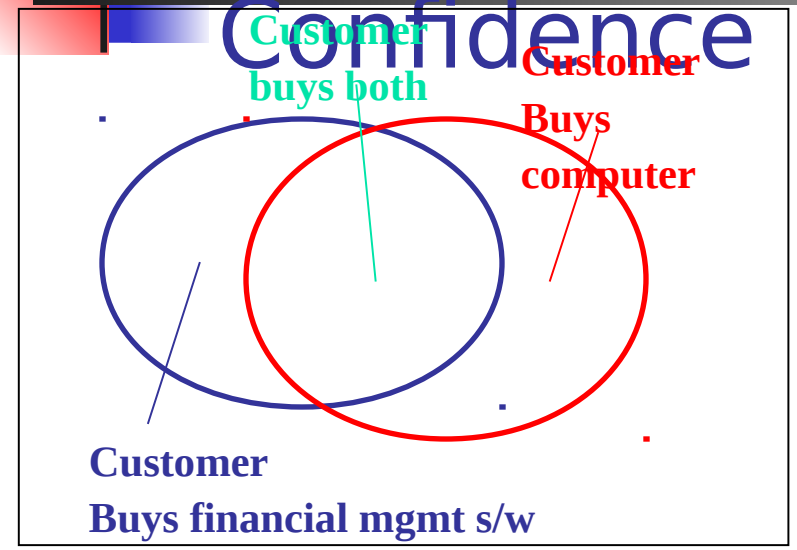
major(x, "CS") ^ takes(x, "DB") $\rightarrow$ grade(x, "A")

[1%, 75%]

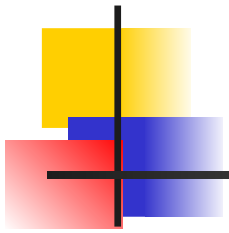
Association Rule: Basic Concepts

- Given: (1) database of transactions, (2) each transaction is a list of items (purchased by a customer in a visit)
- Find: all rules that correlate the presence of one set of items with that of another set of items
  - E.g., *98% of people who purchase tires and auto accessories also get automotive services done*
- Applications
  - * ⇒ *Maintenance Agreement* (What the store should do to boost Maintenance Agreement sales)
  - *Home Electronics* ⇒ * (What other products should the store stocks up?)
  - Attached mailing in direct marketing

Rule Measures: Support and Confidence



- Find all the rules $X \& Y \Rightarrow Z$ with minimum confidence and support
  - support, *s*, probability that a transaction contains {X Y Z}
  - confidence, *c*, conditional probability that a transaction having {X Y} also contains *Z*



Transaction ID	Items Bought
2000	A,B,C
1000	A,C
4000	A,D
5000	B,E,F

Let minimum support 50%, and minimum confidence 50%, we have

- $A \Rightarrow C$ (50%, 66.6%)
- $C \Rightarrow A$ (50%, 100%)

Association Rule Mining: A Road Map

- Boolean vs. quantitative associations (Based on the types of values handled)
  - buys(x, “SQLServer”) ^ buys(x, “DMBook”) → buys(x, “DBMiner”) [0.2%, 60%]
  - age(x, “30..39”) ^ income(x, “42..48K”) → buys(x, “PC”) [1%, 75%]
- Single dimension vs. multiple dimensional associations (see ex. Above)
- Single level vs. multiple-level analysis
  - Items brought are referenced at different levels of abstraction.
  - age(x, “30..39”) => buys(x, “laptop computer”)
  - age(x, “30..39”) => buys(x, “computer”)

Chapter 6: Mining Association Rules in Large Databases

- Association rule mining
- Mining single-dimensional Boolean association rules from transactional databases
- Mining multilevel association rules from transactional databases
- Mining multidimensional association rules from transactional databases and data warehouse
- From association mining to correlation analysis
- Constraint-based association mining
- Summary

Mining Association Rules—An Example

Transaction ID	Items Bought
2000	A,B,C
1000	A,C
4000	A,D
5000	B,E,F

Min. support 50%
Min. confidence 50%

Frequent Itemset	Support
{A}	75%
{B}	50%
{C}	50%
{A,C}	50%

For rule $A \Rightarrow C$:

support = support({A C}) = 50%

confidence = support({A C})/support({A}) = 66.6%

The Apriori principle:

Any subset of a frequent itemset must be frequent

Mining Frequent Itemsets: the Key Step

- Find the *frequent itemsets*: the sets of items that have minimum support
  - A subset of a frequent itemset must also be a frequent itemset
    - i.e., if {*AB*} is a frequent itemset, both {*A*} and {*B*} should be a frequent itemset
  - Iteratively find frequent itemsets with cardinality from 1 to *k (k*-itemset*)*
- Use the frequent itemsets to generate association rules.

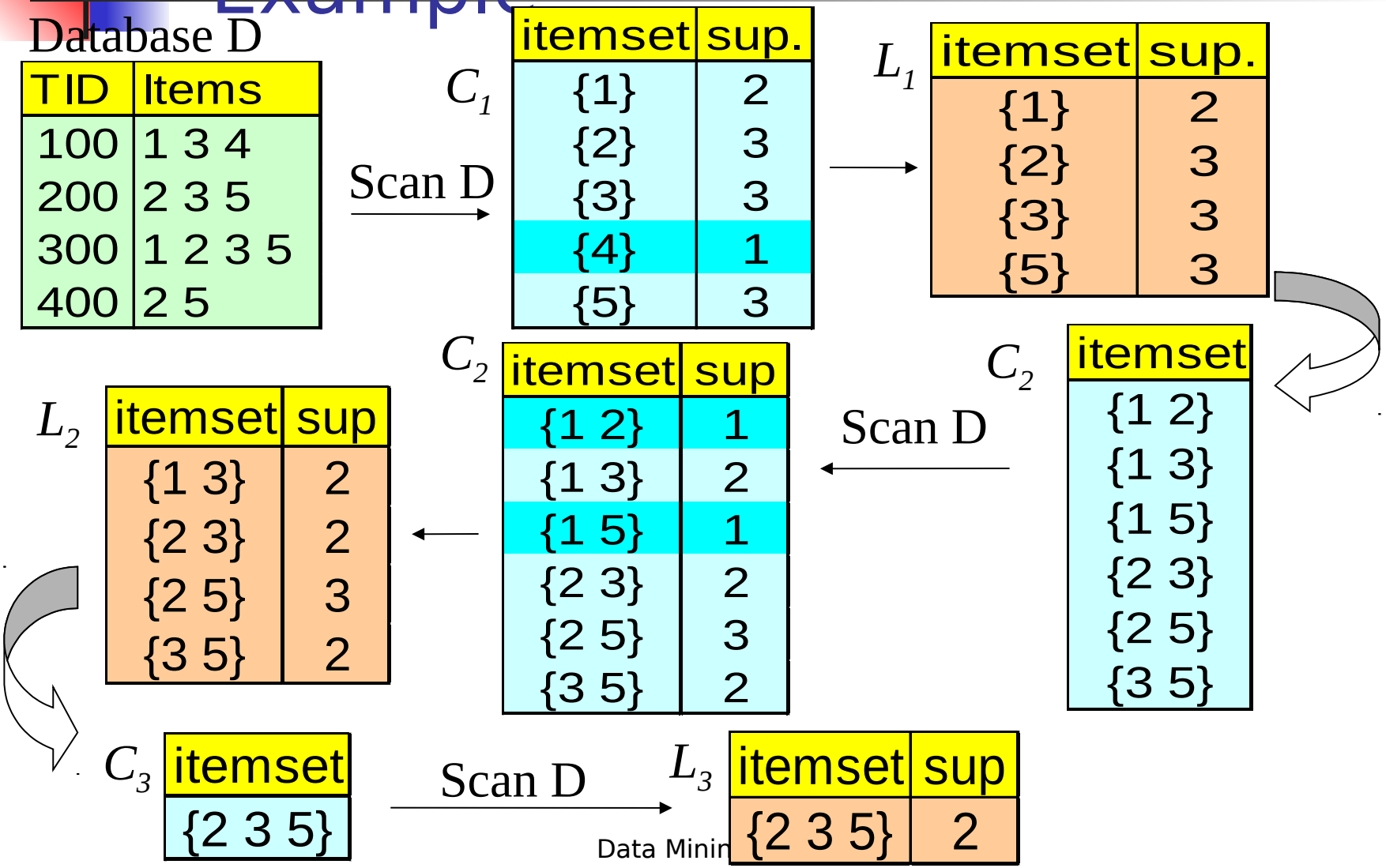
The Apriori Algorithm

- Join Step: C_k is generated by joining $L_{k-1}$with itself
- Prune Step: Any (k-1)-itemset that is not frequent cannot be a subset of a frequent k-itemset
- Pseudo-code:

C_k: Candidate itemset of size k
L_k : frequent itemset of size k

L_1 = {frequent items};
for (k = 1; L_k !=∅; k++) **do begin**
  C_{k+1} = candidates generated from L_k;
  for each transaction t in database do
    increment the count of all candidates in C_{k+1} that are contained in t
  L_{k+1} = candidates in C_{k+1} with min_support
  end
return $\cup_k L_k$;

The Apriori Algorithm — Example



How to Generate Candidates?

- Suppose the items in L_{k-1} are listed in an order
- Step 1: self-joining L_{k-1}

  insert into $\boldsymbol{C_k}$

  select $\boldsymbol{p.item_1, p.item_2, \ldots, p.item_{k-1}, q.item_{k-1}}$

  from $\boldsymbol{L_{k-1}\ p, L_{k-1}\ q}$

  where $\boldsymbol{p.item_1=q.item_1, \ldots, p.item_{k-2}=q.item_{k-2},}$
  $\boldsymbol{p.item_{k-1} < q.item_{k-1}}$

- Step 2: pruning

  forall ***itemsets c in*** $\boldsymbol{C_k}$ do

  forall ***(k-1)-subsets s of c*** do

  if *(s is not in* L_{k-1}*)* **then delete** *c* **from** C_k

How to Count Supports of Candidates?

- Why counting supports of candidates a problem?
  - The total number of candidates can be very huge
  - One transaction may contain many candidates
- Method:
  - Candidate itemsets are stored in a *hash-tree*
  - *Leaf* node of hash-tree contains a list of itemsets and counts
  - *Interior* node contains a hash table
  - *Subset function*: finds all the candidates contained in a transaction

Example of Generating Candidates

- L_3={*abc, abd, acd, ace, bcd*}
- Self-joining: L_3*L_3
  - *abcd* from *abc* and *abd*
  - *acde* from *acd* and *ace*
- Pruning:
  - *acde* is removed because *ade* is not in L_3
- C_4={*abcd*}

Methods to Improve Apriori's Efficiency

- Hash-based itemset counting: A *k*-itemset whose corresponding hashing bucket count is below the threshold cannot be frequent
- Transaction reduction: A transaction that does not contain any frequent k-itemset is useless in subsequent scans
- Partitioning: Any itemset that is potentially frequent in DB must be frequent in at least one of the partitions of DB
- Sampling: mining on a subset of given data, lower support threshold + a method to determine the completeness
- Dynamic itemset counting: add new candidate itemsets only when all of their subsets are estimated to be frequent

Is Apriori Fast Enough? — Performance Bottlenecks

- The core of the Apriori algorithm:
  - Use frequent (k – 1)-itemsets to generate candidate frequent k-itemsets
  - Use database scan and pattern matching to collect counts for the candidate itemsets
- The bottleneck of *Apriori*: candidate generation
  - Huge candidate sets:
    - 10^4 frequent 1-itemset will generate 10^7 candidate 2-itemsets
    - To discover a frequent pattern of size 100, e.g., $\{a_1, a_2, \ldots, a_{100}\}$, one needs to generate $2^{100} \approx 10^{30}$ candidates.
  - Multiple scans of database:
    - Needs (n +1) scans, n is the length of the longest pattern

Mining Frequent Patterns Without Candidate Generation

- Compress a large database into a compact, Frequent-Pattern tree (FP-tree) structure
  - highly condensed, but complete for frequent pattern mining
  - avoid costly database scans
- Develop an efficient, FP-tree-based frequent pattern mining method
  - A divide-and-conquer methodology: decompose mining tasks into smaller ones
  - Avoid candidate generation: sub-database test only!

Construct FP-tree from a Transaction DB

TID	Items bought	(ordered) frequent items
100	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
200	{a, b, c, f, l, m, o}	{f, c, a, b, m}
300	{b, f, h, j, o}	{f, b}
400	{b, c, k, s, p}	{c, b, p}
500	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}

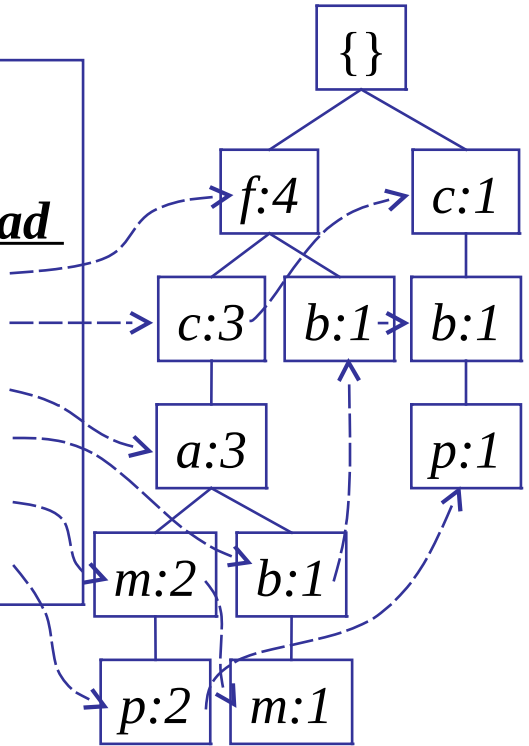
min_support = 0.5

Steps:

1. Scan DB once, find frequent 1-itemset (single item pattern)
2. Order frequent items in frequency descending order
3. Scan DB again, construct FP-tree

Header Table

Item	frequency	head
f	4	
c	4	
a	3	
b	3	
m	3	
p	3	

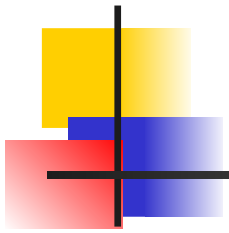


Benefits of the FP-tree Structure

- Completeness:
  - never breaks a long pattern of any transaction
  - preserves complete information for frequent pattern mining
- Compactness
  - reduce irrelevant information—infrequent items are gone
  - frequency descending ordering: more frequent items are more likely to be shared
  - never be larger than the original database (if not count node-links and counts)
  - Example: For Connect-4 DB, compression ratio could be over 100

Mining Frequent Patterns Using FP-tree

- General idea (divide-and-conquer)
  - Recursively grow frequent pattern path using the FP-tree
- Method
  - For each item, construct its conditional pattern-base, and then its conditional FP-tree
  - Repeat the process on each newly created conditional FP-tree
  - Until the resulting FP-tree is empty, or it contains only one path (single path will generate all the combinations of its sub-paths, each of which is a frequent pattern)

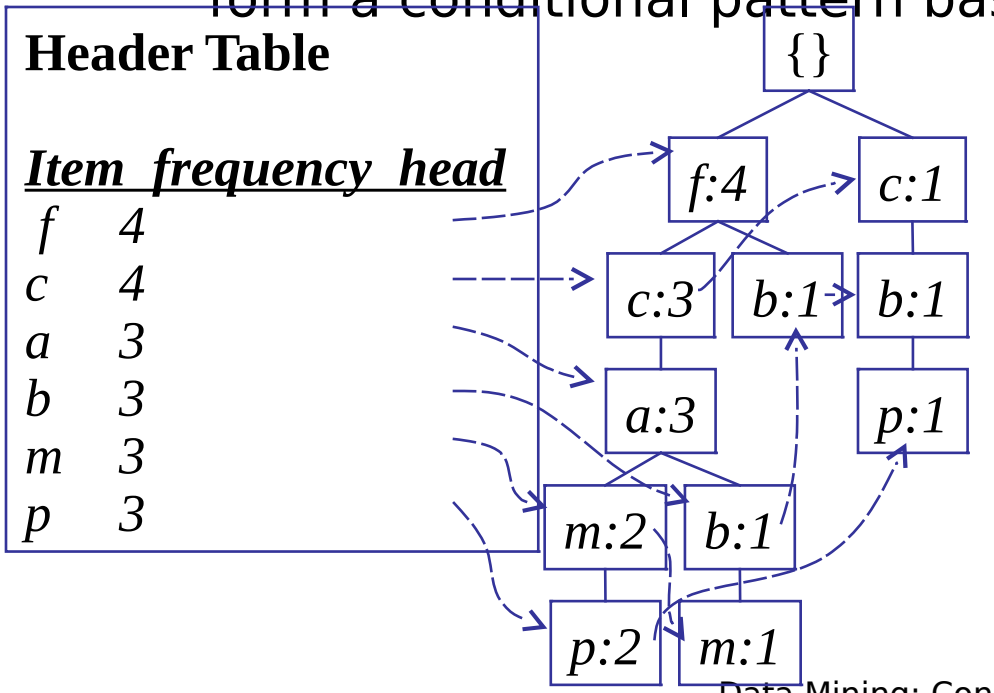


Major Steps to Mine FP-tree

1) Construct conditional pattern base for each node in the FP-tree
2) Construct conditional FP-tree from each conditional pattern-base
3) Recursively mine conditional FP-trees and grow frequent patterns obtained so far
   - If the conditional FP-tree contains a single path, simply enumerate all the patterns

Step 1: From FP-tree to Conditional Pattern Base

- Starting at the frequent header table in the FP-tree
- Traverse the FP-tree by following the link of each frequent item
- Accumulate all of transformed prefix paths of that item to form a conditional pattern base



***Conditional* pattern bases**

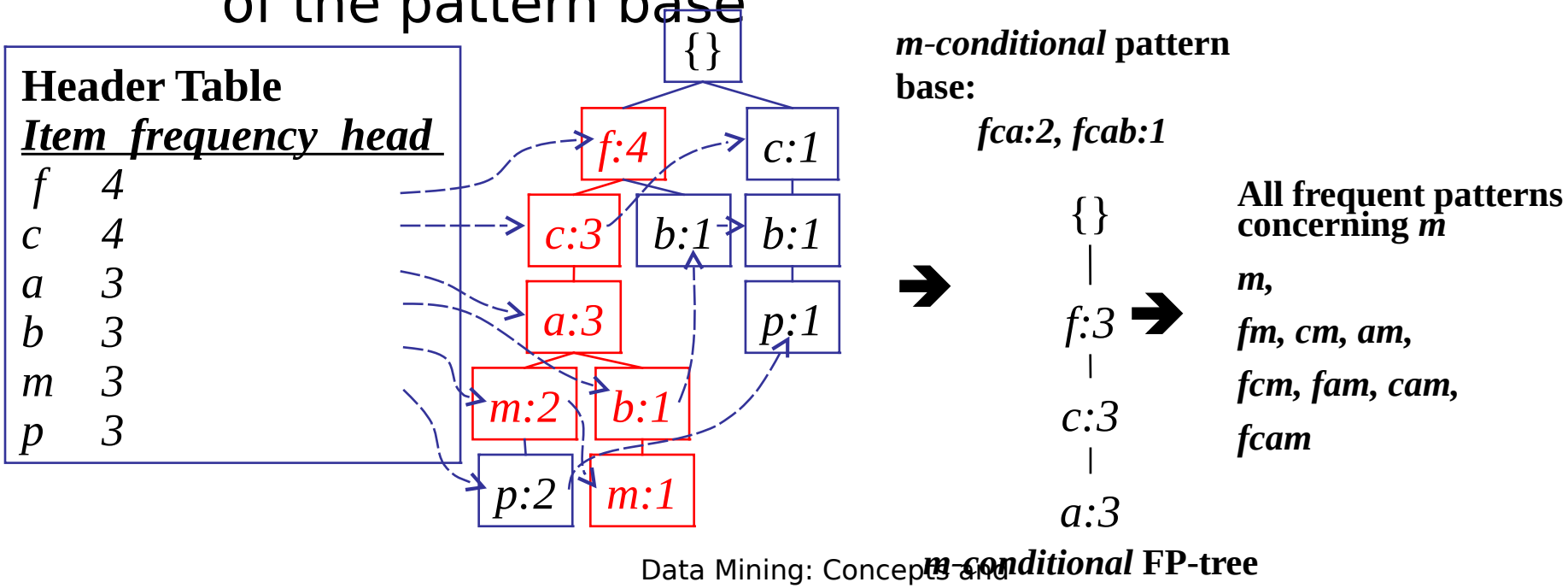
item	***cond. pattern base***
c	***f:3***
a	***fc:3***
b	***fca:1, f:1, c:1***
m	***fca:2, fcab:1***
p	***fcam:2, cb:1***

Properties of FP-tree for Conditional Pattern Base Construction

- Node-link property
  - For any frequent item a_i, all the possible frequent patterns that contain a_i can be obtained by following a_i's node-links, starting from a_i's head in the FP-tree header
- Prefix path property
  - To calculate the frequent patterns for a node a_i in a path P, only the prefix sub-path of a_i in P need to be accumulated, and its frequency count should carry the same count as node a_i.

Step 2: Construct Conditional FP-tree

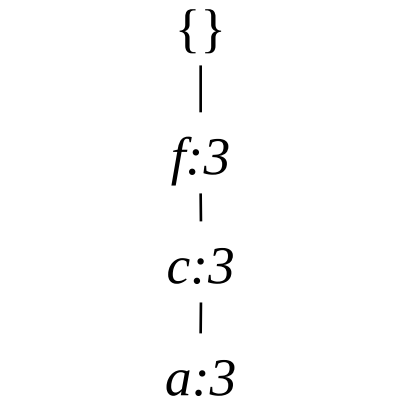
- For each pattern-base
  - Accumulate the count for each item in the base
  - Construct the FP-tree for the frequent items of the pattern base



Mining Frequent Patterns by Creating Conditional Pattern-Bases

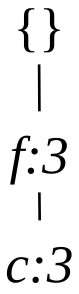
Item	Conditional pattern-base	Conditional FP-tree
p	{(fcam:2), (cb:1)}	{(c:3)}|p
m	{(fca:2), (fcab:1)}	{(f:3, c:3, a:3)}|m
b	{(fca:1), (f:1), (c:1)}	Empty
a	{(fc:3)}	{(f:3, c:3)}|a
c	{(f:3)}	{(f:3)}|c
f	Empty	Empty

Step 3: Recursively mine the conditional FP-tree



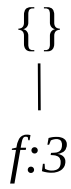
***m-conditional* FP-tree**

Cond. pattern base of “am”: (fc:3)



***am-conditional* FP-tree**

Cond. pattern base of “cm”: (f:3)



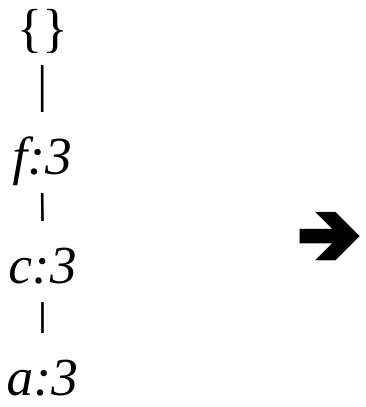
***cm-conditional* FP-tree**

Cond. pattern base of “cam”: (f:3)

***cam-conditional* FP-tree**

Single FP-tree Path Generation

- Suppose an FP-tree T has a single path P
- The complete set of frequent pattern of T can be generated by enumeration of all the combinations of the sub-paths of P



➔

All frequent patterns concerning *m*

m,

fm, cm, am,

fcm, fam, cam,

fcam

***m-conditional* FP-tree**

Principles of Frequent Pattern Growth

- Pattern growth property
  - Let α be a frequent itemset in DB, B be α's conditional pattern base, and β be an itemset in B. Then $\alpha \cup \beta$ is a frequent itemset in DB iff β is frequent in B.
- "*abcdef* " is a frequent pattern, if and only if
  - "*abcde* " is a frequent pattern, and
  - "*f* " is frequent in the set of transactions containing "*abcde* "

Why Is Frequent Pattern Growth Fast?

- Our performance study shows
  - FP-growth is an order of magnitude faster than Apriori, and is also faster than tree-projection
- Reasoning
  - No candidate generation, no candidate test
  - Use compact data structure
  - Eliminate repeated database scan
  - Basic operation is counting and FP-tree building

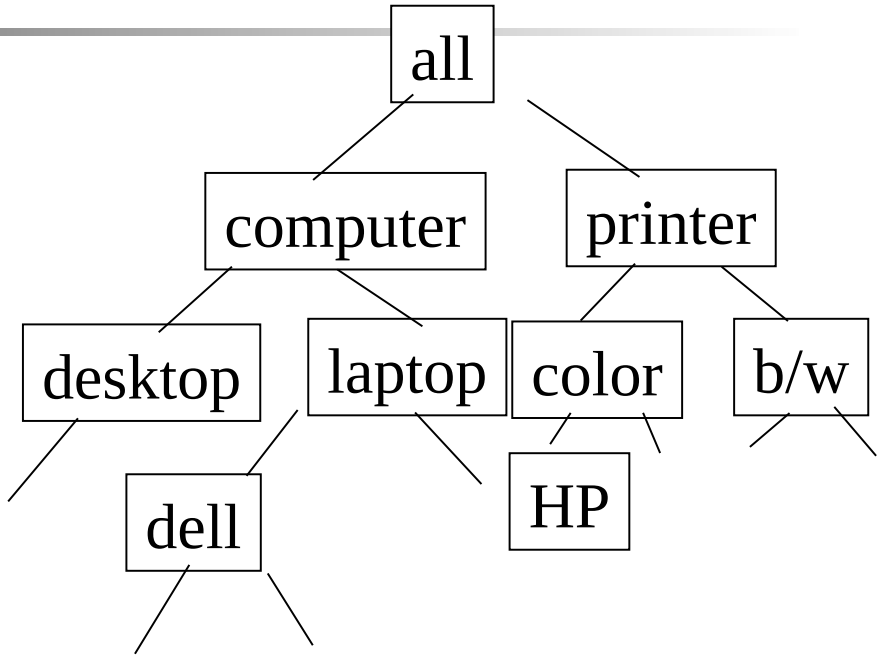
Chapter 6: Mining Association Rules in Large Databases

- Association rule mining
- Mining single-dimensional Boolean association rules from transactional databases
- Mining multilevel association rules from transactional databases
- Mining multidimensional association rules from transactional databases and data warehouse
- From association mining to correlation analysis
- Constraint-based association mining
- Summary

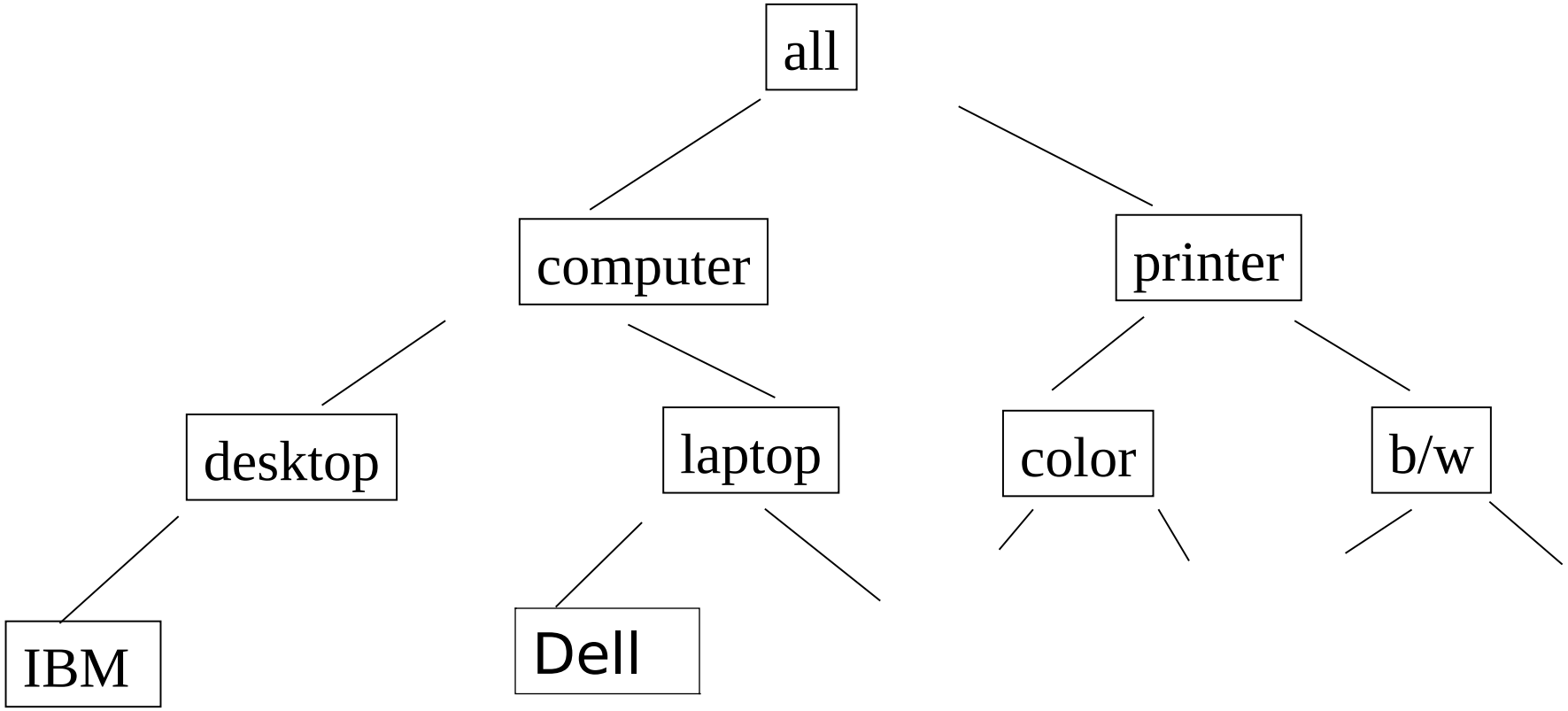
Multiple-Level Association Rules

- Items often form hierarchy.
- Items at the lower level are expected to have lower support.
- Rules regarding itemsets at

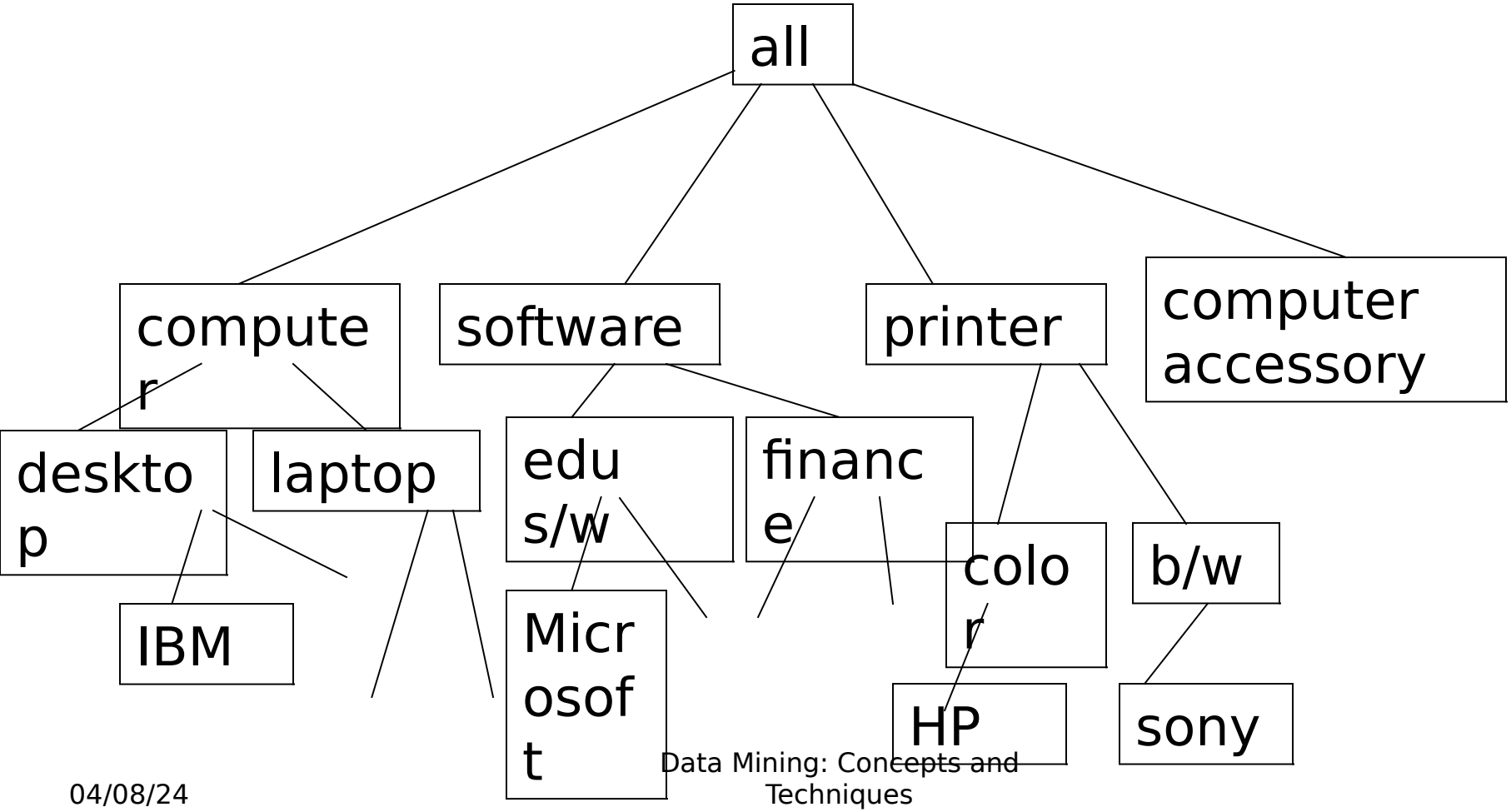
  appropriate levels could be quite useful.
- Transaction database can be encoded based on dimensions and levels
- We can explore shared



A concept Hierarchy



A concept Hierarchy



Mining Multi-Level Associations

- A top_down, progressive deepening approach:
  - First find high-level strong rules:
    computer → printer [20%, 60%].
  - Then find their lower-level “weaker” rules:
    laptop computer → color printer [6%, 50%].

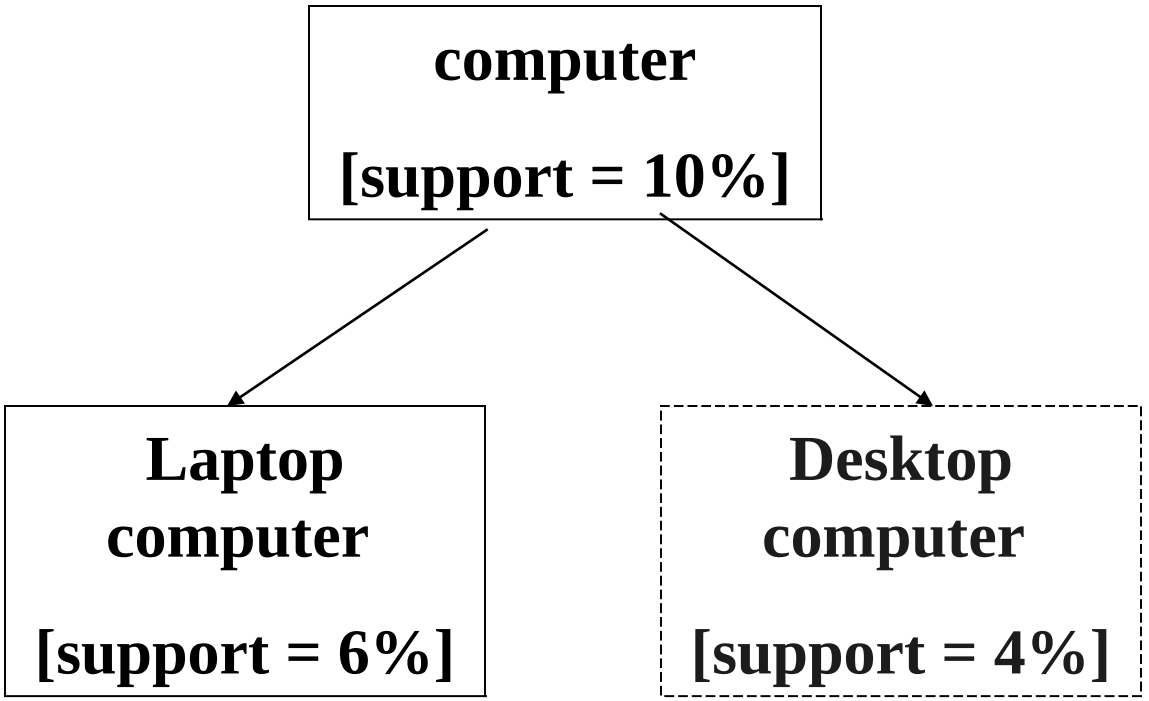
Multi-level Association: Uniform Support vs. Reduced Support

- Uniform Support: the same minimum support for all levels
  - + One minimum support threshold. No need to examine itemsets containing any item whose ancestors do not have minimum support.
  - – Lower level items do not occur as frequently. If support threshold
    - too high ⇒ miss low level associations
    - too low ⇒ generate too many high level associations
- Reduced Support: reduced minimum support at lower levels
  - There are 4 search strategies:
    - Level-by-level independent
    - Level-cross filtering by k-itemset
    - Level-cross filtering by single item
    - Controlled level-cross filtering by single item

Uniform Support

Multi-level mining with uniform support

Level 1
min_sup = 5%



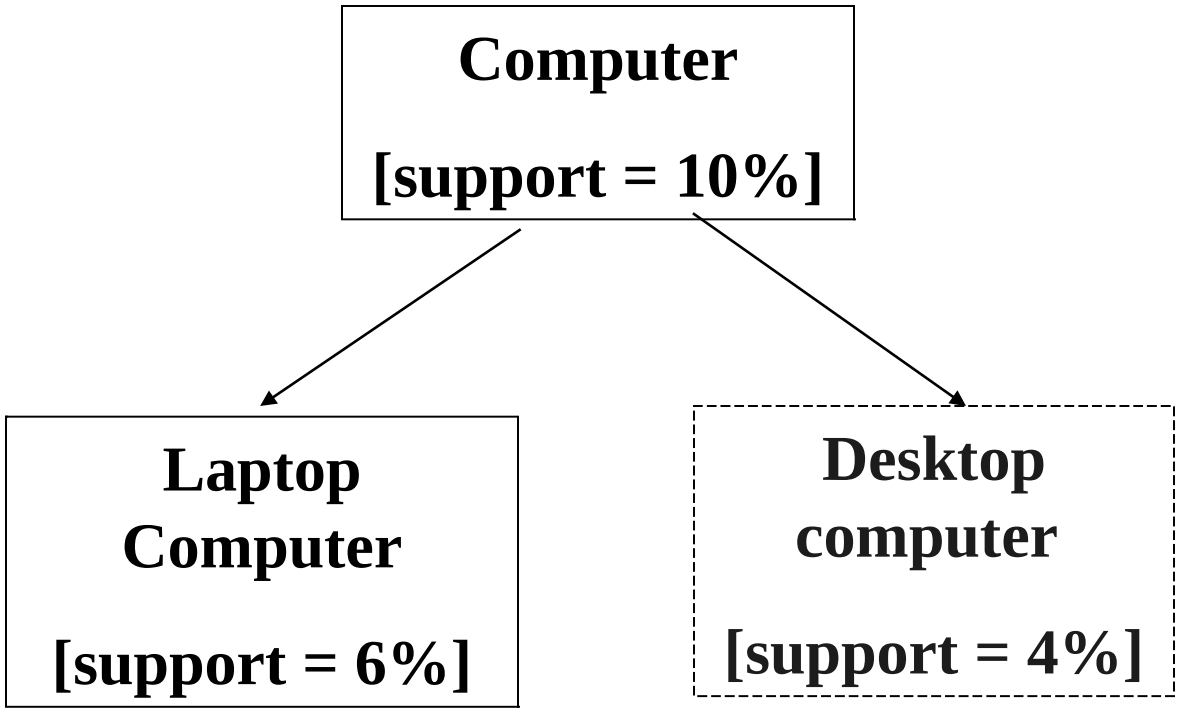
Level 2
min_sup = 5%

Back

Reduced Support

Level 1
min_sup = 12%

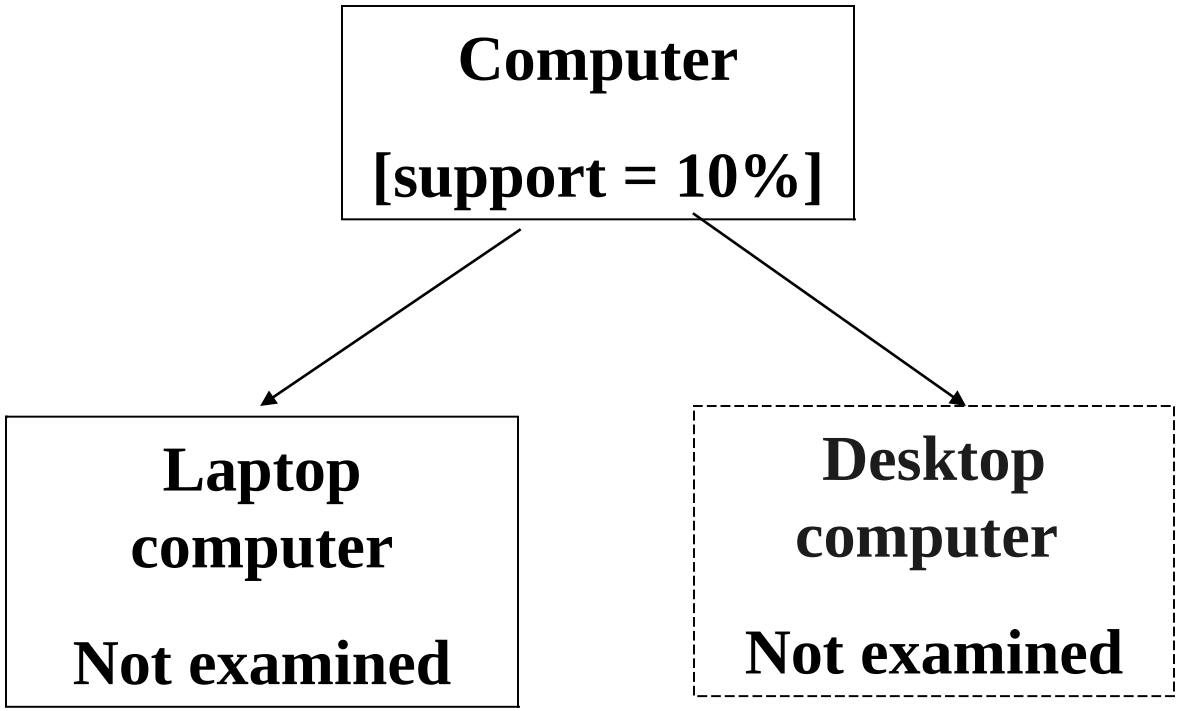
Level 2
min_sup = 3%



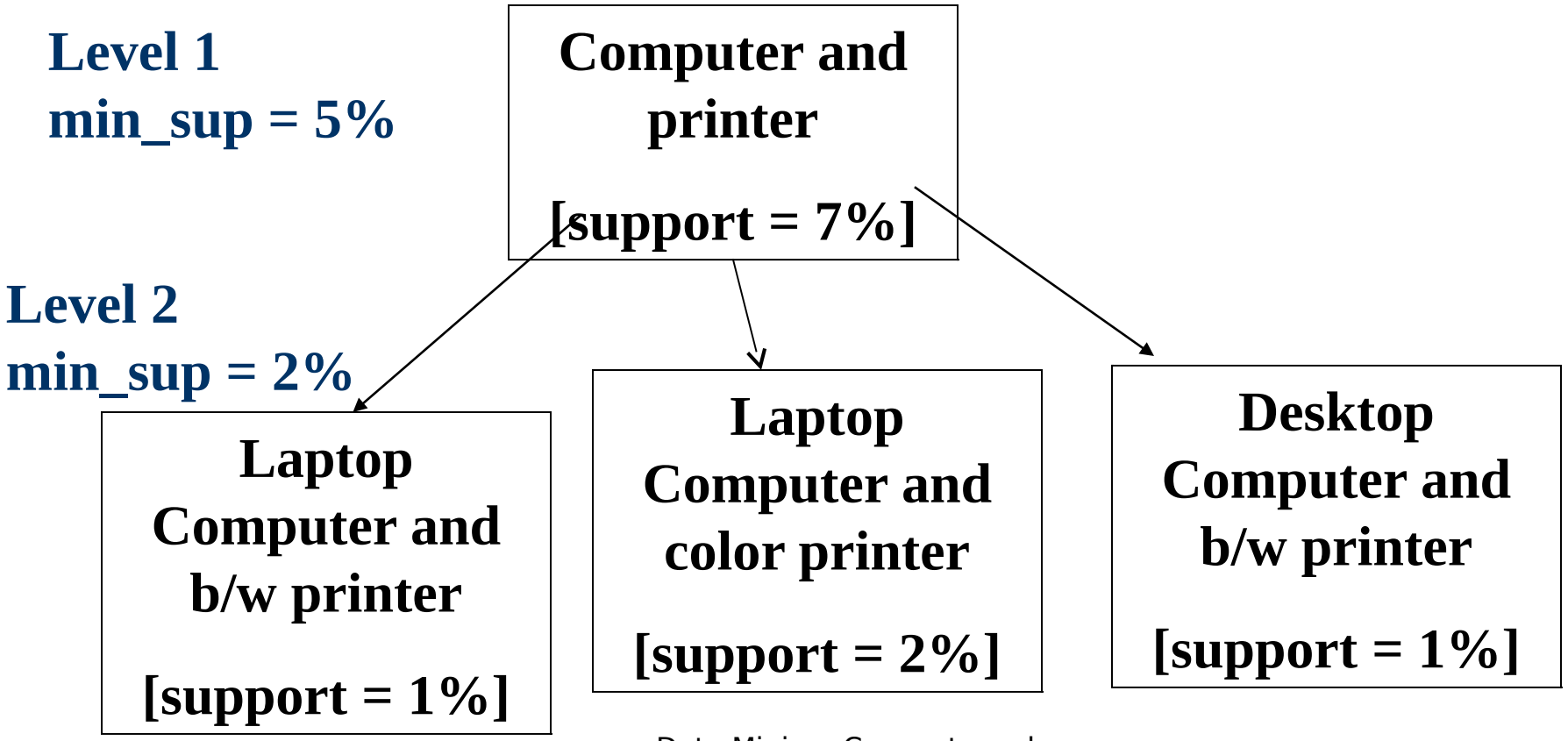
Level cross filtering by a single item

Level 1
min_sup = 12%

Level 2
min_sup = 3%



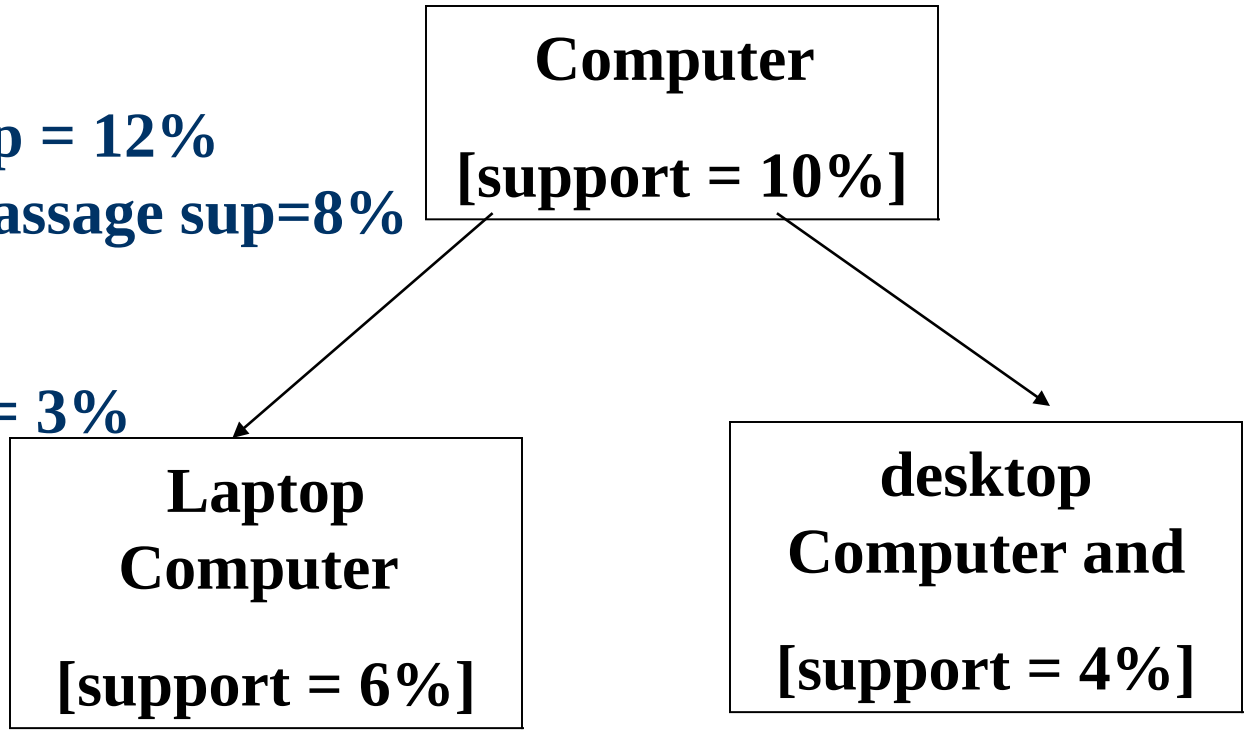
Level cross filtering by k-itemset



Controlled level cross filtering by single item

Level 1
min_sup = 12%
Level passage sup=8%

Level 2
min_sup = 3%



Cross level association rules

- Example
  - “computer ⇒ b/w printer” where items within the rule are not required to belong to the same concept level.
  - Ex of same level association rules
  - “computer ⇒ printer”
  - “desktop computer ⇒ b/w printer”

Multi-level Association: Redundancy Filtering

- Some rules may be redundant due to “ancestor” relationships between items.
- Example
  - Desktop computer ⇒ b/w printer [support = 8%, confidence = 70%]
  - IBM desktop computer ⇒ b/w printer [support = 2%, confidence = 72%]
- We say the first rule is an ancestor of the second rule.
- A rule is redundant if its support is close to the “expected” value, based on the rule’s ancestor.

Chapter 6: Mining Association Rules in Large Databases

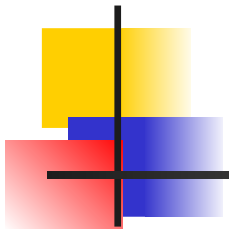
- Association rule mining
- Mining single-dimensional Boolean association rules from transactional databases
- Mining multilevel association rules from transactional databases
- Mining multidimensional association rules from transactional databases and data warehouse
- From association mining to correlation analysis
- Constraint-based association mining
- Summary

Multi-Dimensional Association: Concepts

Single-dimensional rules(Intra dimension association rule)

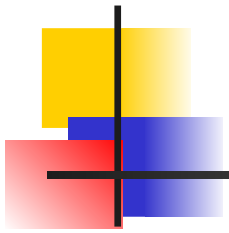
buys(X, “IBM desktop computer”) ⇒ buys(X, “sony b/w printer”)

- Multi-dimensional rules: ▯ 2 dimensions or predicates
  - Inter-dimension association rules (*no repeated predicates*)age(X,”19-25”) ∧ occupation(X,“student”) ⇒ buys(X,“laptop”)
  - hybrid-dimension association rules (*repeated predicates*)
    age(X,”20-29”) ∧ buys(X, “laptop”) ⇒ buys(X, “b/w printer”)



Types of Attributes

- Categorical Attributes (nominal attributes)
  - finite number of possible values, no ordering among values (e.g. occupation, brand ,color)
- Quantitative Attributes
  - numeric, implicit ordering among values (e.g. age,income,price)



Techniques for Mining MD Associations

- Search for frequent *k*-predicate set:
  - Example: {age, occupation, buys} is a 3-predicate set.
  - Techniques can be categorized by how age are treated.

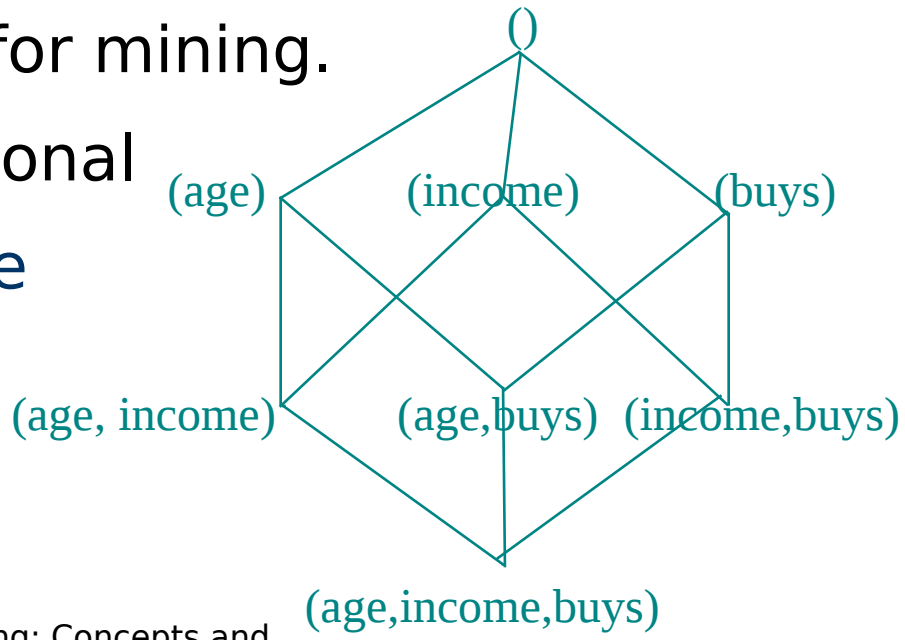
1. Using static discretization of quantitative attributes
   - Quantitative attributes are statically discretized by using predefined concept hierarchies.
2. Quantitative association rules
   - Quantitative attributes are dynamically discretized into “bins”based on the distribution of the data.

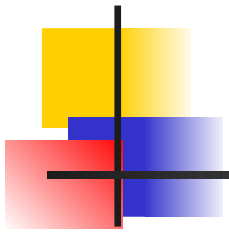
Distance-based association rules

This is a dynamic discretization process that considers the distance between data points.

Static Discretization of Quantitative Attributes

- Discretized prior to mining using concept hierarchy.
- Numeric values are replaced by ranges.
- In relational database, finding all frequent k-predicate sets will require *k* or *k*+1 table scans.
- Data cube is well suited for mining.
- The cells of an n-dimensional cuboid correspond to the predicate sets.
- Mining from data cubes can be much faster.





Quantitative Association Rules

- Numeric attributes are *dynamically* discretized
  - Such that the confidence or compactness of the rules mined is maximized.

- 2-D quantitative association rules:

$$A_{quan1} \wedge A_{quan2} \Rightarrow A_{cat}$$

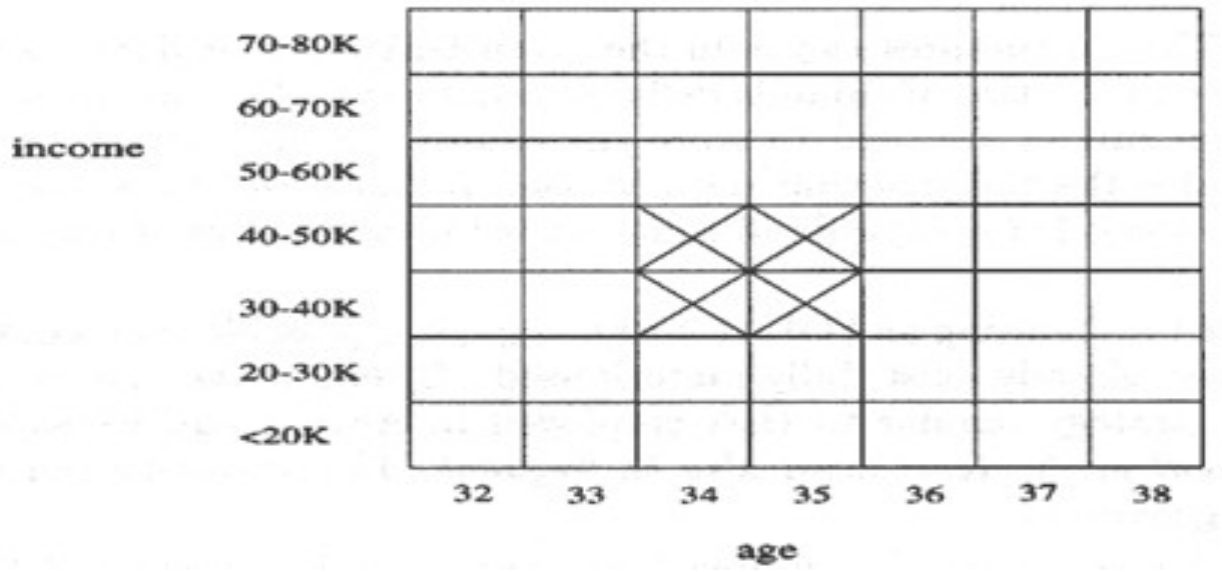
age(X,”30..39”) ∧ income(X,”42k …48K”) ⇒ buys(X, resolution TV”)

age(X,34) ∧ income(X,”31K - 40K”) ⇒ buys(X,”high resolution TV”)

age(X,35) ∧ income(X,”31K - 40K”) ⇒ buys(X,”high resolution TV”)

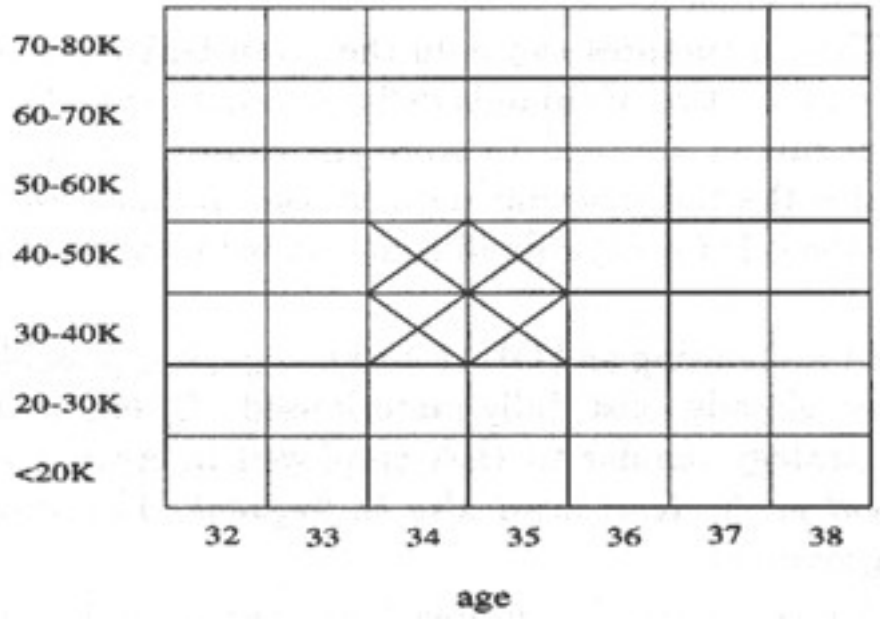
age(X,34) ∧ income(X,”41K - 50K”) ⇒ buys(X,”high resolution TV”)

age(X,35) ∧ income(X,”41K - 50K”) ⇒ buys(X,”high resolution TV”)



Clustering the Association Rules

- Cluster "adjacent" association rules to form general rules using a 2-D grid.



age(X,"34..35") ∧ income(X,"31k …50K") ⇒ buys(X, resolution TV")

ARCS (Association Rule Clustering System)

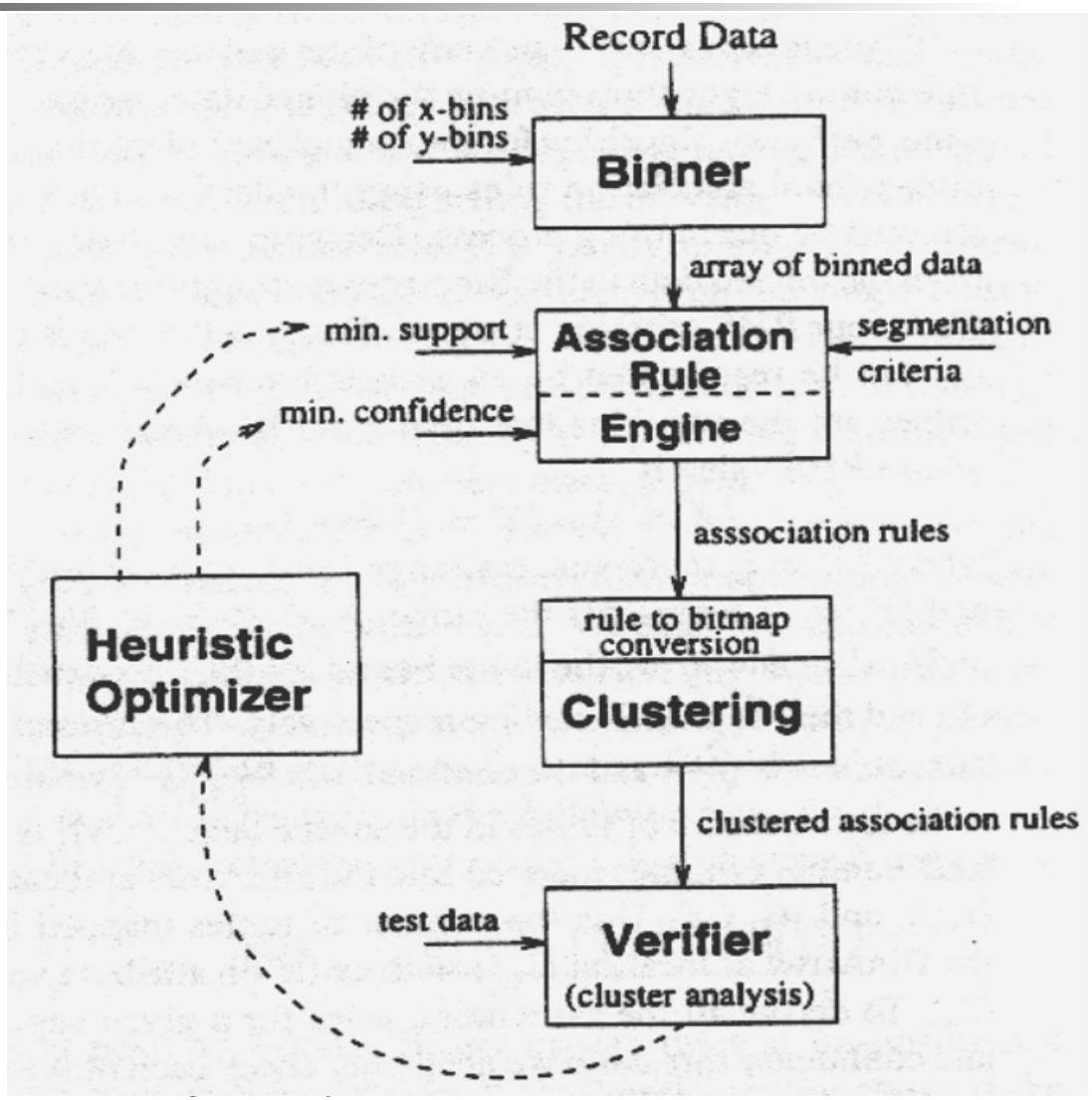
How does ARCS work?

1. Binning

2. Find frequent predicateset

3. Clustering

4. Optimize



Limitations of ARCS

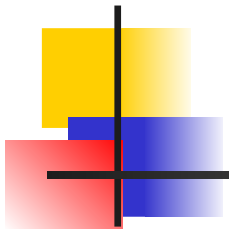
- Only quantitative attributes on LHS of rules.
- Only 2 attributes on LHS. (2D limitation)
- An alternative to ARCS
  - Non-grid-based
  - equi-depth binning
  - clustering based on a measure of *partial completeness*.

Mining Distance-based Association Rules

- Binning methods do not capture the semantics of interval data

Price($)	Equi-width (width $10)	Equi-depth (depth 2)	Distance-based
7	[0,10]	[7,20]	[7,7]
20	[11,20]	[22,50]	[20,22]
22	[21,30]	[51,53]	[50,53]
50	[31,40]		
51	[41,50]		
53	[51,60]		

- Distance-based partitioning, more meaningful discretization considering:
  - density/number of points in an interval
  - "closeness" of points in an interval



Mining Distance-based Association Rules

- Equidepth partitioning -groups distant values together.
- Equiwidth partitioning - split values that are close together and create intervals for which there is no data.
- Distance based partitioning – considers the density or number of points in an interval,closeness of points in an interval.

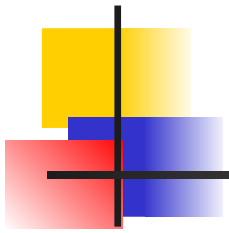
Approximation in data values

- **Item_type(x,"electronic")^ manufacturer(x,"foreign") ⇒ price(X,$200)**
- **Association rules don't allow for approximation of attribute values. Support and confidence measures do not consider the closeness of values for a given attribute.**
- **That is why distance based association rules are required.**

Mining Distance-based Association Rules

- A 2 phase algorithm is used to mine distance based association rules.
- The first employs clustering to find the intervals or clusters.
- The second phase obtains distance based association rules by searching for group of clusters that occur frequently together.
- **Example of distance based association rule**

$$Cx \Rightarrow Cy$$



Mining Distance-based Association Rules

- Ex. Cx cluster for age and Cy cluster for income.
- When age clustered tuples Cx are projected onto the attribute income , their corresponding income values lie within the income cluster Cy or close to it.
- The distance measures the degree of association between Cx and Cy. The smaller the distance stronger the association is.

Clusters and Distance Measurements

- S[X] is a set of N tuples t_1, t_2, …, t_N, projected on the attribute set X
- The diameter of S[X]:

$$d(S[X]) = \frac{\sum\sum distX(ti[X], tj[X])}{N(N-1)}$$

  - $dist_x$:distance metric, e.g. Euclidean distance or Manhattan

Clusters and Distance Measurements(Cont.)

- The diameter, *d*, assesses the density of a cluster C_X, where

$$d(CX) \leq d0^X$$

$$|CX| \geq s0$$

- Finding clusters and distance-based rules
  - the density threshold, d_0, replaces the notion of support
  - modified version of the BIRCH clustering algorithm

Chapter 6: Mining Association Rules in Large Databases

- Association rule mining
- Mining single-dimensional Boolean association rules from transactional databases
- Mining multilevel association rules from transactional databases
- Mining multidimensional association rules from transactional databases and data warehouse
- From association mining to correlation analysis
- Constraint-based association mining
- Summary

Interestingness Measurements

- Objective measures

  Two popular measurements:

  1. *support;* and
  2. *confidence*

- Subjective measures (Silberschatz & Tuzhilin, KDD95)

  A rule (pattern) is interesting if

  1. it is *unexpected* (surprising to the user); and/or
  2. *actionable* (the user can do something with it)

Criticism to Support and Confidence

- Example 1: (Aggarwal & Yu, PODS98)
  - Among 5000 students
    - 3000 play basketball
    - 3750 eat cereal
    - 2000 both play basket ball and eat cereal
  - *play basketball* $\Rightarrow$ *eat cereal* [40%, 66.7%] is misleading because the overall percentage of students eating cereal is 75% which is higher than 66.7%.
  - *play basketball* $\Rightarrow$ *not eat cereal* [20%, 33.3%] is far more accurate, although with lower support and confidence

	basketball	not basketball	sum(row)
cereal	2000	1750	3750
not cereal	1000	250	1250
sum(col.)	3000	2000	5000

lift

$$lift = \frac{P(A \cup B)}{P(A)P(B)}$$

$$lift(B,C) = \frac{2000/5000}{3000/5000 * 3750/5000} = 0.89$$

$$lift(B,\neg C) = \frac{1000/5000}{3000/5000 * 1250/5000} = 1.33$$

Criticism to Support and Confidence (Cont.)

- Example 2:
  - X and Y: positively correlated,
  - X and Z, negatively related
  - support and confidence of X=>Z dominates
- We need a measure of dependent or correlated events

$$corr_{A,B} = \frac{P(A \cup B)}{P(A)P(B)}$$

X	1	1	1	1	0	0	0	0
Y	1	1	0	0	0	0	0	0
Z	0	1	1	1	1	1	1	1

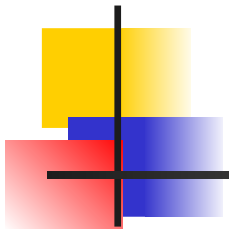
Rule	Support	Confidence
X=>Y	25%	50%
X=>Z	37.50%	75%

Other Interestingness Measures: Interest

- Interest (correlation, lift) $\frac{P(A \wedge B)}{P(A)P(B)}$
  - taking both P(A) and P(B) in consideration
  - P(A^B)=P(B)*P(A), if A and B are independent events
  - A and B negatively correlated, if the value is less than 1; otherwise A and B positively correlated

X	1	1	1	1	0	0	0	0
Y	1	1	0	0	0	0	0	0
Z	0	1	1	1	1	1	1	1

Itemset	Support	Interest
X,Y	25%	2
X,Z	37.50%	0.9
Y,Z	12.50%	0.57



Chapter 6: Mining Association Rules in Large Databases

- Association rule mining
- Mining single-dimensional Boolean association rules from transactional databases
- Mining multilevel association rules from transactional databases
- Mining multidimensional association rules from transactional databases and data warehouse
- From association mining to correlation analysis
- Constraint-based association mining
- Summary

Constraint-Based Mining

- Interactive, exploratory mining giga-bytes of data?
  - Could it be real? — Making good use of constraints!
- What kinds of constraints can be used in mining?
  - Knowledge type constraint: classification, association, etc.
  - Data constraint: SQL-like queries
    - Find product pairs sold together in Vancouver in Dec.'98.
  - Dimension/level constraints:
    - in relevance to region, price, brand, customer category.
  - Rule constraints
    - small sales (price < $10) triggers big sales (sum > $200).
  - Interestingness constraints:

Rule Constraints in Association Mining

- Two kind of rule constraints:
  - Rule form constraints: meta-rule guided mining.
    - P(x, y) ^ Q(x, w) $\rightarrow$ takes(x, “database systems”).
  - Rule (content) constraint: constraint-based query optimization.
    - sum(LHS) < 100 ^ min(LHS) > 20 ^ count(LHS) > 3 ^ sum(RHS) > 1000
- 1-variable vs. 2-variable constraints:
  - 1-var: A constraint confining only one side (L/R) of the rule, e.g., as shown above.
  - 2-var: A constraint confining both sides (L and R).
    - sum(LHS) < min(RHS) ^ max(RHS) < 5* sum(LHS)

Constrain-Based Association Query

- Database: (1) trans (TID, Itemset), (2) itemInfo (Item, Type, Price)
- A constrained asso. query (CAQ) is in the form of $\{(S_1, S_2) | C\}$,
  - where C is a set of constraints on S_1, S_2 including frequency constraint
- A classification of (single-variable) constraints:
  - Class constraint: $S \subset A$. *e.g.* $S \subset Item$
  - Domain constraint:
    - $S \theta v$, $\theta \in \{=, \neq, <, \leq, >, \geq\}$. *e.g.* $S.Price < 100$
    - $v \theta S$, θ is $\in$ or $\notin$. *e.g.* $snacks \notin S.Type$
    - $V \theta S$, or $S \theta V$, $\theta \in \{\subseteq, \subset, \not\subset, =, \neq\}$
      - *e.g.* $\{snacks, sodas\} \subseteq S.Type$
  - Aggregation constraint: $agg(S)\ \theta\ v$, where *agg* is in $\{min, max, sum, count, avg\}$, and $\theta \in \{=, \neq, <, \leq, >, \geq\}$.
    - *e.g.* $count(S_1.Type) = 1$, $avg(S_2.Price) < 100$

Constrained Association Query Optimization Problem

- Given a CAQ = { (S_1, S_2) | C }, the algorithm should be :
  - sound: It only finds frequent sets that satisfy the given constraints C
  - complete: All frequent sets satisfy the given constraints C are found
- A naïve solution:
  - Apply Apriori for finding all frequent sets, and then to test them for constraint satisfaction one by one.
- Our approach:
  - Comprehensive analysis of the properties of constraints and try to push them as deeply as possible inside the frequent set computation.

Anti-monotone and Monotone Constraints

- A constraint C_a is anti-monotone iff. for any pattern S not satisfying C_a, none of the super-patterns of S can satisfy C_a
- A constraint C_m is monotone iff. for any pattern S satisfying C_m, every super-pattern of S also satisfies it

Succinct Constraint

- A subset of item I_s is a succinct set, if it can be expressed as $\sigma_p(I)$ for some selection predicate p, where σ is a selection operator
- $SP \subseteq 2^I$ is a succinct power set, if there is a fixed number of succinct set $I_1, \ldots, I_k \subseteq I$, s.t. SP can be expressed in terms of the strict power sets of $I_1, \ldots, I_k$ using union and minus
- A constraint C_s is succinct provided

Convertible Constraint

- Suppose all items in patterns are listed in a total order R
- A constraint C is **convertible anti-monotone** iff a pattern S satisfying the constraint implies that each suffix of S w.r.t. R also satisfies C
- A constraint C is **convertible monotone** iff a pattern S satisfying the constraint implies that each pattern of which S is a suffix w.r.t. R also satisfies C

Relationships Among Categories of Constraints